

Arquitectura de Datos

Tema 3: Redis

Bases de datos clave-valor en memoria

Curso académico 2026/2027



T3.4 Persistencia y distribución



Resultados de aprendizaje

- a. Comprender las estrategias de persistencia: RDB, AOF y modo híbrido
- b. Conocer la arquitectura de replicación primario/réplica
- c. Entender Sentinel como sistema de detección de fallos y failover automático
- d. Comprender Redis Cluster: hash slots, particionado y redirección MOVED

Índice de contenidos

T3.4.1 Persistencia en Redis	T3.4.5 Redis Cluster
T3.4.2 Expulsión de datos	T3.4.6 Decisiones de arquitectura
T3.4.3 Replicación en Redis	T3.4.7 Monitorización y operación
T3.4.4 Redis Sentinel	

T3.4.1 Persistencia en Redis

Aunque Redis es un sistema en memoria, ofrece mecanismos de **persistencia** para garantizar la supervivencia de los datos ante reinicios del servidor.

fork(): Es una llamada al sistema operativo que crea una copia del proceso Redis. El proceso hijo ejecuta BGSAVE (escribe el snapshot a disco) mientras el padre sigue sirviendo comandos normalmente. Gracias a COW, la copia es casi instantánea (<1ms) porque solo se copian los metadatos de las páginas, no los datos.

Contenido avanzado: La persistencia en Redis es importante para producción, pero para entender los conceptos básicos basta con saber que Redis ofrece RDB (snapshots) y AOF (log). El modo híbrido (RDB+AOF) es la recomendación actual.

RDB (Redis Database) — Snapshots

RDB crea **snapshots** (instantáneas) periódicas de todo el dataset en un fichero comprimido (dump.rdb). El proceso utiliza fork() para crear un proceso hijo que escribe el dataset en disco. Con copy-on-write, el proceso hijo tiene una copia de la tabla de páginas, así que el padre sigue procesando comandos mientras el hijo escribe el snapshot. **Precaución:** en datasets grandes (>10 GB), el fork() puede causar un pico de latencia porque copia la tabla de páginas completa.

Copy-on-Write (COW): fork() crea una copia del proceso Redis, pero NO copia la memoria físicamente. El hijo comparte las mismas páginas de memoria con el padre hasta que uno de ellos escribe. Al escribir, el SO crea una copia solo de esa página modificada. Ventaja: el snapshot no bloquea el servidor. Riesgo: si el padre escribe mucho durante el snapshot, la memoria puede duplicarse (cada escritura crea una copia de la página). En producción, monitoriza INFO memory durante BGSAVE.

Configuración de RDB

```
# redis.conf
save 60 1000    # Snapshot si 1000 keys cambiaron en 60 s
save 300 10     # Snapshot si 10 keys cambiaron en 5 min
save 60 1       # Snapshot si 1 key cambió en 1 min

# Desactivar RDB
save ""

# Guardar manualmente
BGSAVE    # Background save (fork – no bloqueante)
SAVE      # Síncrono (bloquea el servidor – ¡no usar en producción!)
```

Ventajas de RDB

- Rápido para backups y restauraciones.
- Bajo overhead en producción.

Desventajas de RDB

- Puede perder datos entre snapshots (últimos N segundos).

- Buena compatibilidad con réplicas.
- Formato compacto (fichero comprimido).
- Ideal para disaster recovery.

- El fork puede bloquear en datasets grandes (>10GB).
- No es ideal para datos críticos por sí solo.
- El archivo RDB es un binario, no legible.

AOF (Append Only File) — Log de escrituras

AOF registra **cada operación de escritura** en un fichero de log. Es más seguro que RDB pero genera ficheros más grandes.

Configuración de AOF

```
# redis.conf
appendonly yes
appendfilename "appendonly.aof"

# Estrategia de sincronización del SO
appendfsync everysec    # Cada segundo (recomendado – balance seguridad/rendimiento)
appendfsync always      # En cada escritura (máxima seguridad – más lento)
appendfsync no          # Dejar que el SO decida (más rápido – menos seguro)

# Reescritura automática del AOF (compactar)
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
```

Ventajas de AOF

- Menor pérdida de datos (máximo 1 segundo con everysec).
- Fichero legible por humanos (formato Redis).
- Recuperación más granular.
- No bloquea el servidor durante la escritura normal (con appendfsync everysec).

Desventajas de AOF

- Ficheros más grandes que RDB.
- Recuperación más lenta.
- Mayor overhead en escritura.
- Reescritura puede causar picos de I/O.

Ejemplo real de archivo AOF: Así se ve un archivo AOF (formato RESP legible por humanos):

```
*3
$3
SET
$4
foo
$3
bar
*3
$3
INCR
$7
counter
*3
$3
SET
$4
foo
$6
bar2
```

Cada línea es un comando Redis. El AOF es un log secuencial de todas las escrituras. Para reconstruir el estado, Redis replay los comandos desde el inicio.

Línea temporal de pérdida de datos:

```
RDB:    |---snapshot---|---snapshot---|---FAIL---| → Pierde hasta 15 min (si snapshot cada 15 min)
AOF:    |log continuo.....|FAIL| → Pierde hasta 1s (si fsync everysec)
Híbrido:|log continuo.....|FAIL| → RDB para backup + AOF para mínimo loss
```

Modo híbrido (recomendado en producción)

Redis 4.0+ permite usar **RDB + AOF simultáneamente**. El AOF se regenera periódicamente a partir del último snapshot RDB.

Modo híbrido

```
# redis.conf
aof-use-rdb-preamble yes # AOF empieza con snapshot RDB
```

Twitter usa modo híbrido RDB+AOF para sus clústeres Redis. RDB permite backups rápidos (copiar dump.rdb), mientras que AOF garantiza que no se pierden más de 1 segundo de datos ante un fallo del servidor.

Caso real — GitHub: GitHub migró de MySQL a Redis para su sistema de contadores (stars, forks, watchers).

La decisión fue:

- **RDB para snapshots:** Un dump completo cada 24 horas. Si Redis cae, se pierde como mucho 24h de contadores (que son aproximados, no críticos).
- **AOF con appendfsync everysec:** Cada star/fork se registra en el log AOF. En el peor caso, se pierde 1 segundo de datos (unas pocas estrellas).
- **Reconstrucción periódica:** Una vez al día, GitHub reconstruye los contadores desde MySQL (la fuente de verdad) y los carga en Redis. Así, si Redis pierde datos, la reconstrucción los restaura.

Lección: Para contadores aproximados, RDB+AOF es suficiente. No necesitas consistencia perfecta si el dato es "cuántas estrellas tiene un repo" y no "cuánto dinero tiene una cuenta".

Resumen T3.4.1: RDB: snapshots rápidos pero puede perder datos. AOF: log seguro pero fichero grande. Híbrido (recomendado): RDB+AOF. appendfsync everysec es el mejor balance.

T3.4.2 Expulsión de datos

Redis es un sistema en memoria: cuando la RAM se llena, expulsa datos según la política configurada (`maxmemory-policy`).

Las 8 políticas de evicción (LRU, LFU, random, TTL, etc.) y sus casos de uso se describen en detalle en T3.3

§Políticas de expulsión. En esta sección nos centramos en cómo **monitorizar** la evicción en producción: `INFO stats` muestra `evicted_keys` (número de claves eliminadas por falta de memoria). Si `evicted_keys` crece constantemente, necesitas más memoria o un TTL más corto.

Monitorización de memoria

```
INFO memory
# used_memory: 1073741824      (1 GB)
# used_memory_rss: 1157627904  (RAM del SO)
# mem_fragmentation_ratio: 1.08 (fragmentación = rss/used)
# maxmemory: 2147483648       (2 GB)
# maxmemory_policy: allkeys-lru

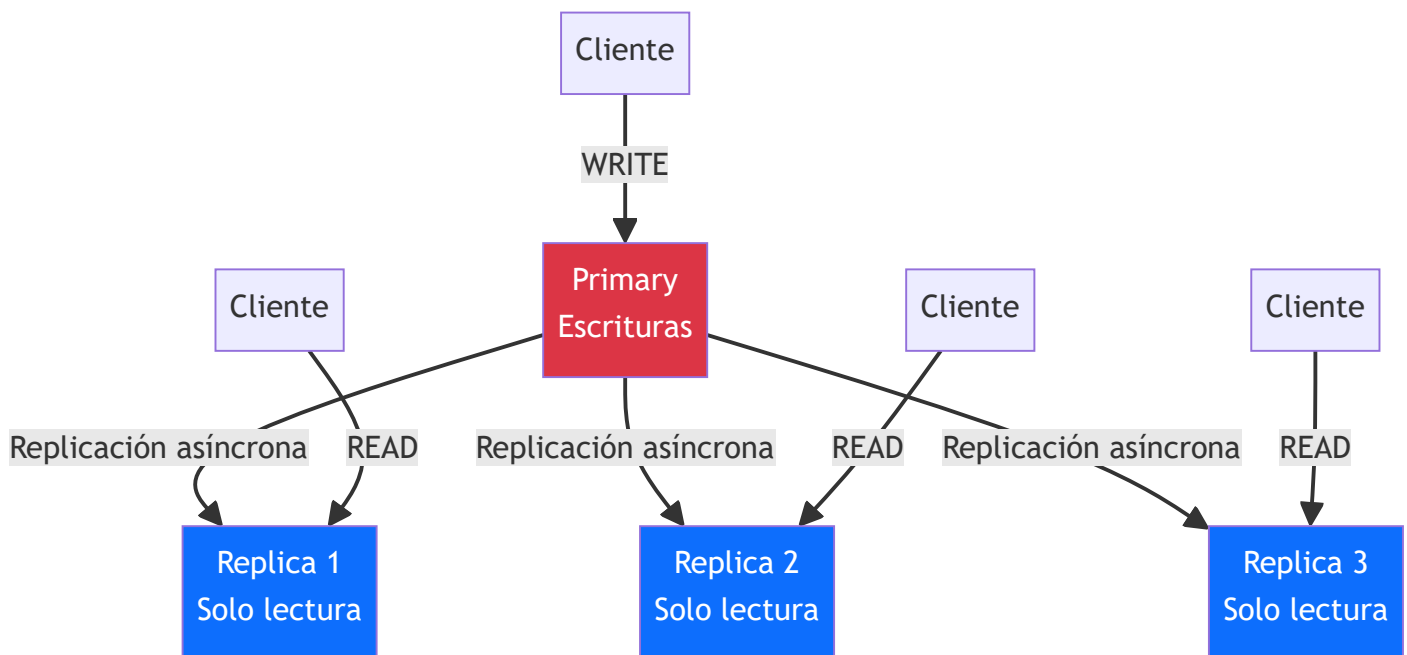
# Si fragmentation_ratio > 1.5 → hay mucha memoria desperdiciada
# Si used_memory ≈ maxmemory → se están expulsando claves
```

Twitch usa Redis para caché de chat; cuando la memoria se llena, expulsa colores de chat de usuarios inactivos manteniendo los datos más recientes de canales activos.

Resumen T3.4.2: `maxmemory` configura el límite. `maxmemory-policy` determina qué se expulsa. Monitoriza `used_memory` y `evicted_keys` con `INFO memory`.

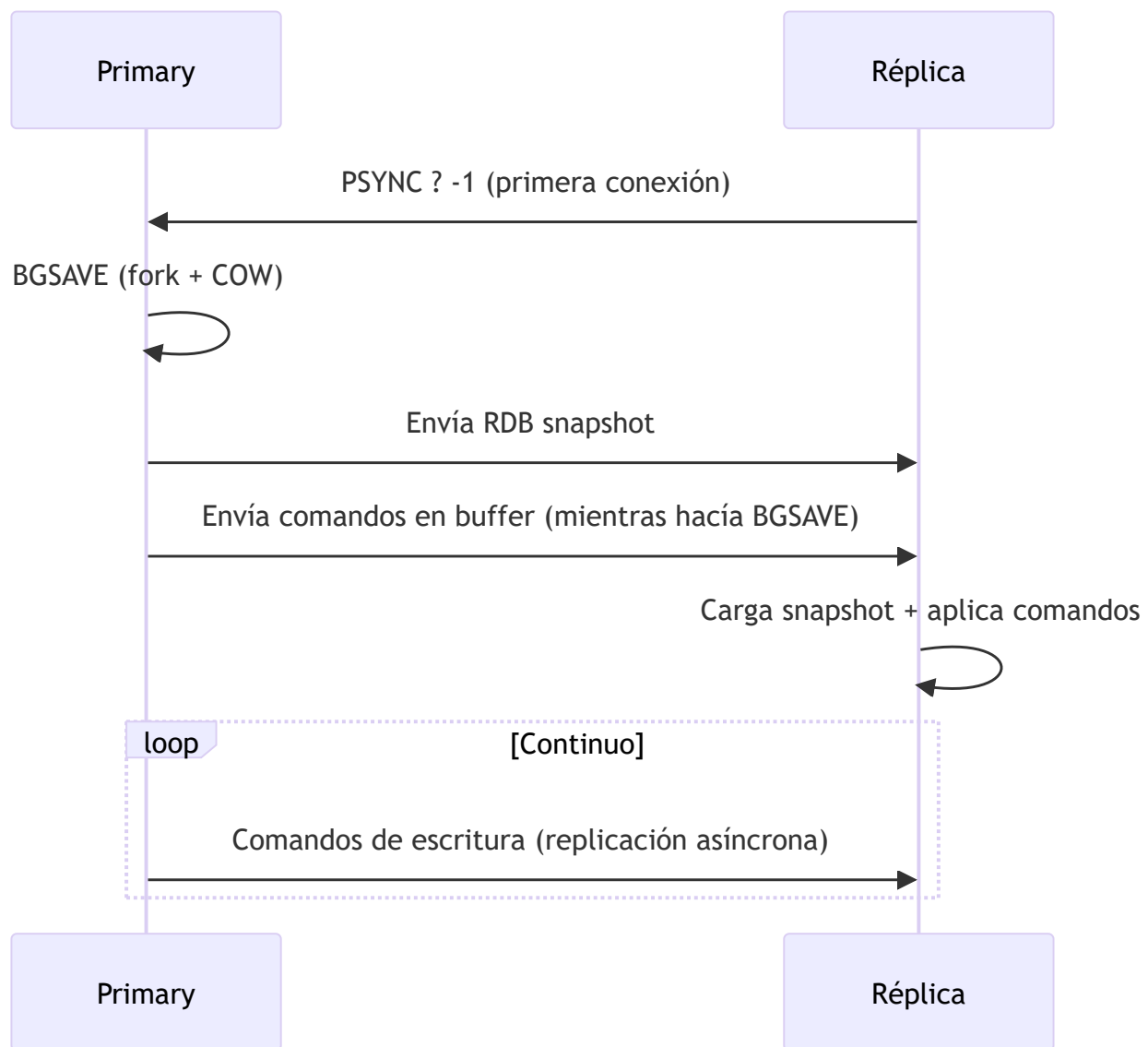
T3.4.3 Replicación en Redis

La replicación en Redis sigue una arquitectura **primary-replica**: un nodo primario recibe todas las escrituras y uno o más réplicas mantienen copias de solo lectura.



Flujo de replicación

1. El replica se conecta al primary y envía PSYNC.
2. El primary ejecuta BGSAVE y envía el snapshot al replica.
3. Mientras se envía el snapshot, el primary almacena nuevos comandos en un buffer.
4. Una vez enviado el snapshot, el primary envía los comandos del buffer.
5. A partir de ahí, el primary envía cada comando de escritura al replica de forma asíncrona.



- **Ventajas:** Alta disponibilidad, lectura escalable, backup automático.
- **Limitaciones:** El primary es el único punto de escritura. La replicación es asíncrona (puede haber lag de milisegundos).

Analogía: El primario es como el original de un documento y las réplicas son las copias. Cada vez que modificas el original, las copias se actualizan. Si el original se pierde, una copia se promueve a original. La diferencia es que en Redis las copias pueden tener un ligero desfase (replicación asíncrona).

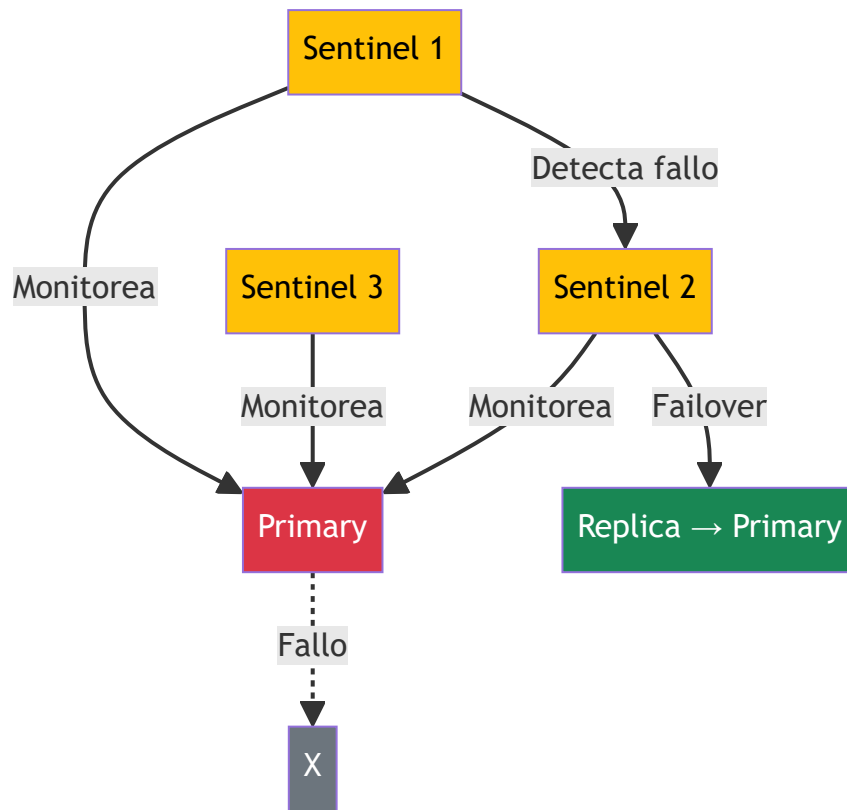
Discord usa replicación Redis con factor 3 para garantizar disponibilidad del caché de chat incluso con fallos de nodo. Si el primary falla, uno de los réplicas asume el rol de primary automáticamente.

Resumen T3.4.3: Primary-replica asíncrona. El primary escribe, los réplicas leen. Buena disponibilidad pero escritura limitada a un nodo. Factor de replicación 3 = tolerancia a 2 fallos.

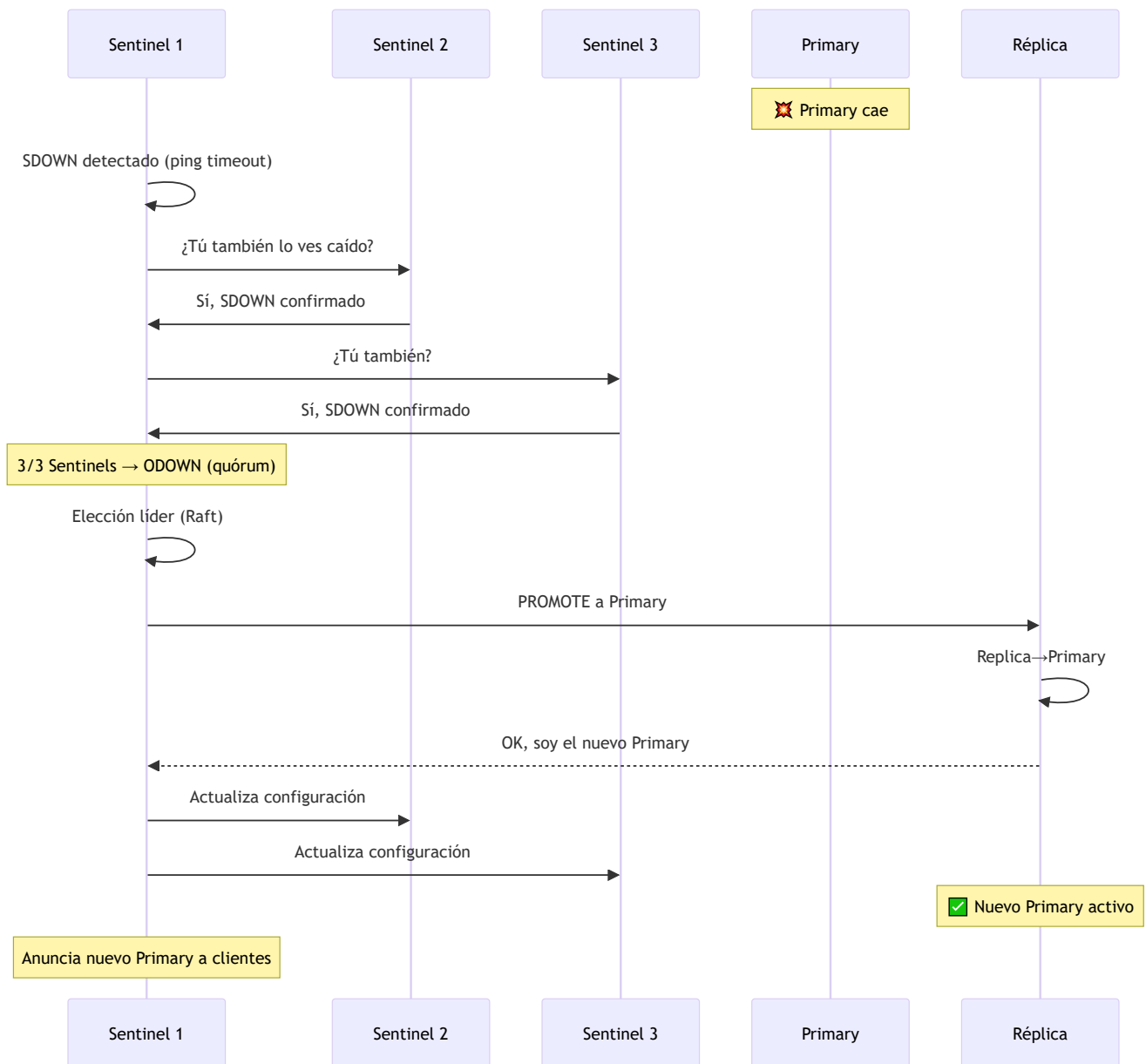
Clúster en producción: Para datos críticos, usa replicación + Sentinel (alta disponibilidad) o Cluster (escalabilidad horizontal). Nunca dependas de un solo nodo de Redis en producción.

T3.4.4 Redis Sentinel

Redis Sentinel es un sistema de **alta disponibilidad** que monitoriza nodos Redis y ejecuta failover automático.



Proceso de failover automático



«La redundancia no es un gasto, es una inversión en disponibilidad.» — En producción, un sistema sin redundancia es un sistema que fallará. Redis Sentinel y Cluster son tu seguro de vida.

Failover en producción: El proceso completo tarda 10-30 segundos. Los clientes pierden conexión temporalmente pero reconectan automáticamente al nuevo Primary. Sentinel es fundamental para alta disponibilidad en Redis.

Funcionamiento:

- Los Sentinels monitorizan el primary cada segundo con PING.
- Si un Sentinel detecta que el primary no responde, lo marca como SDOWN (Subjectively Down).
- Cuando múltiples Sentinels (quórum) coinciden, se marca como ODOWN (Objectively Down).

- Los Sentinels eligen un nuevo primary entre los réplicas y notifican a los clientes.

SDOWN (Subjective Down): Un Sentinel individual detecta que el primary no responde tras el timeout. Es subjetivo: puede ser un problema de red del propio Sentinel, no del primary.

ODOWN (Objective Down): Múltiples Sentinels coinciden en que el primary está caído, alcanzando el quórum. Es objetivo: el fallo está confirmado por el consenso del cluster de Sentinels. Solo cuando se alcanza ODOWN se inicia el failover automático.

Quórum: Número mínimo de Sentinels que deben estar de acuerdo para que una decisión (como failover) se ejecute. Si tienes 3 Sentinels y quórum=2, al menos 2 Sentinels deben confirmar que el primary está caído antes de promover un réplica. Esto evita que un Sentinel aislado (con problemas de red) ejecute un failover fantasma.

Failover: Proceso automático de promover un réplica a primary cuando el primary actual falla. En Redis Sentinel: (1) Sentinels detectan SDOWN, (2) alcanzan quórum → ODOWN, (3) eligen un Sentinel coordinador, (4) el coordinador selecciona el mejor réplica (prioridad, offset de replicación, ID), (5) promueve el réplica a primary, (6) reconfigura las demás réplicas. Tiempo típico: 10-30 segundos.

Qué verías en los logs durante un failover:

```
[01] 12:00:01 # +sdown master mymaster 127.0.0.1 6379    ← SDOWN detectado
[01] 12:00:05 # +odown master mymaster 127.0.0.1 6379    ← ODOWN (quórum alcanzado)
[01] 12:00:05 # Failover authentication initiated
[01] 12:00:06 # +failover-state-select-slave              ← Buscando mejor réplica
[01] 12:00:07 # Promoted slave 127.0.0.1:6380            ← Réplica promovida
[01] 12:00:07 # +switch-master mymaster 127.0.0.1 6380   ← Nuevo primary activo
```

Discord usa Redis Sentinel para gestionar instancias de caché de chat con failover automático, consiguiendo disponibilidad del 99,99%.

Resumen T3.4.4: Sentinel monitoriza nodos y ejecuta failover automático. Mínimo 3 Sentinels para quórum. Detecta SDOWN (subjetivo) → ODOWN (objetivo) → failover.

T3.4.5 Redis Cluster

Redis Cluster permite **distribuir datos entre múltiples nodos** de forma nativa, sin necesidad de proxies externos.

Contenido avanzado: Redis Cluster es avanzado y normalmente lo gestiona un equipo de infraestructura. Para un desarrollador, lo importante es entender que los datos se distribuyen en 16.384 slots y que los clientes inteligentes gestionan las redirecciones automáticamente.

Hash slots y distribución de claves

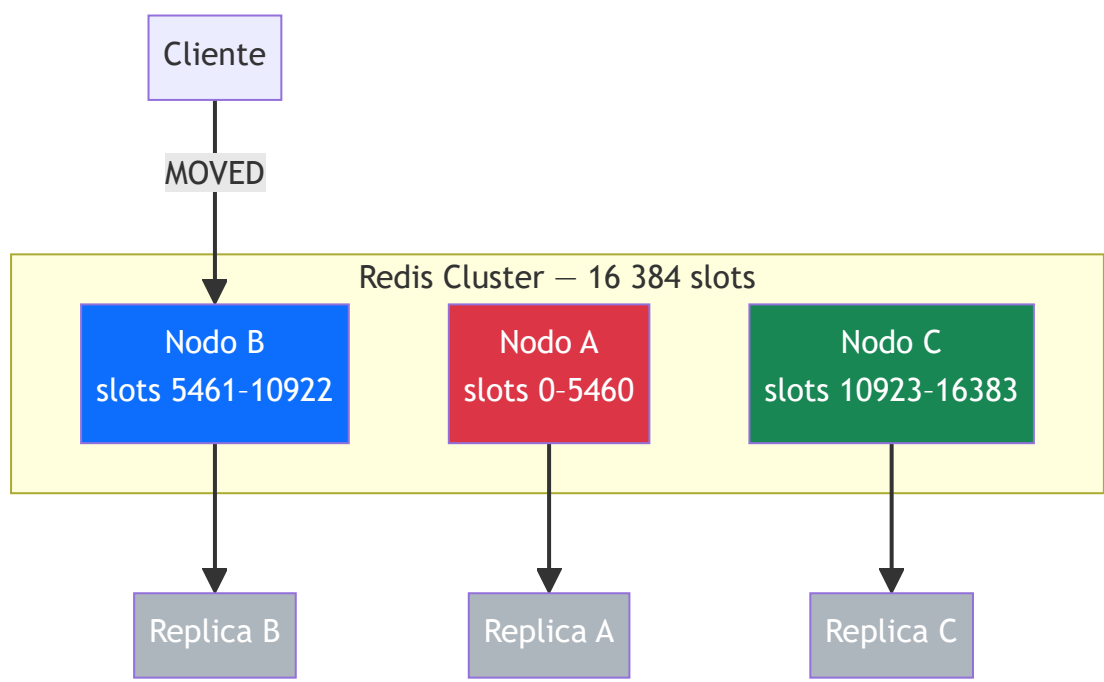
Redis Cluster divide el espacio de claves en **16 384 hash slots**. Cada nodo es responsable de un rango de slots:

Cálculo de hash slots

```
Slot = CRC16(key) % 16384

# Ejemplo con 3 nodos:
# Nodo A: slots 0 - 5460
# Nodo B: slots 5461 - 10922
# Nodo C: slots 10923 - 16383

SET user:1000 "data"    → CRC16("user:1000") % 16384 = 5461 → Nodo B
SET product:456 "data"  → CRC16("product:456") % 16384 = 12300 → Nodo C
SET session:abc "data"  → CRC16("session:abc") % 16384 = 2000 → Nodo A
```



Gossip Protocol: Redis Cluster usa el protocolo Gossip para que cada nodo conozca el estado de los demás sin un coordinator central. Cada segundo, un nodo envía un PING a un nodo aleatorio con su visión del cluster (qué nodos

conoce, qué slots gestiona). El receptor responde con un PONG incluyendo su propia visión. Tras pocas rondas, todos convergen en un estado conocido. Si un nodo no responde tras varios PINGs → marcado como caído.

Redirección -MOVED

Cuando el cliente consulta una clave que está en otro nodo, Redis responde con -MOVED:

Respuesta MOVED

```
-MOVED 5461 192.168.1.2:6379
# El cliente debe reconectar al nodo correcto
# Los clientes inteligentes (Jedis, Lettuce, StackExchange.Redis) gestionan esto automáticamente
```

Resharding en caliente

Redis Cluster permite **mover slots de un nodo a otro sin downtime**. Los slots se migran uno a uno, manteniendo disponibilidad.

Analogía: Redis Cluster es como una biblioteca con 3 salas temáticas. Los 16.384 libros (hash slots) se distribuyen entre las salas. Si pides un libro que está en la sala B pero estás en la sala A, te redirigen con un "MOVED". La distribución es automática y balanceada.

Shard (particionado): Dividir datos entre múltiples instancias Redis, donde cada instancia almacena un subconjunto de claves. En Redis Cluster, las 16384 hash slots se distribuyen entre los nodos. Cada nodo es responsable de un conjunto de slots. Si añades un nodo, solo migras algunos slots (no todos los datos).

¿Por qué 16.384 slots? Es un equilibrio: suficiente granularidad para distribuir carga entre hasta 16.384 nodos, pero no tantos que el protocolo sea ineficiente. Con 3 nodos, cada uno tiene aproximadamente 5.461 slots. Con 1.000 nodos, cada uno tiene aproximadamente 16 slots.

Conexión con T7: Redis Cluster usa hash slots (16.384 fijos). Cassandra usa un token ring con particionado dinámico. Ambos resuelven el mismo problema (distribuir datos entre nodos) pero con estrategias diferentes.

Twitter ejecuta uno de los mayores clústeres Redis del mundo. Originalmente usaban Twemproxy (proxy de sharding) para gestionar miles de instancias. Actualmente Redis Cluster nativo es la opción recomendada para nuevos despliegues.

Resumen T3.4.5: 16.384 hash slots distribuidos entre nodos. Slot = CRC16(key) % 16384. Resharding en caliente sin downtime. Los clientes gestionan MOVED automáticamente.

T3.4.6 Decisiones de arquitectura

Pregunta	Cuándo usar Redis	Cuándo NO usar Redis
¿Qué datos cachear?	Datos de alta frecuencia de acceso (catálogo, sesiones)	Datos que se escriben más de lo que se leen
¿Cuánta memoria necesito?	Caben en RAM disponible del servidor	No caben en memoria (usar BD en disco)
¿Qué consistencia necesito?	Eventualmente consistente (caché)	Consistencia fuerte (banco de datos)
¿Qué latencia necesito?	Sub-milisegundo	Latencia > 1 ms tolerable

Patrón de diseño habitual: Redis como capa de "hot data" delante de una BD persistente.

Netflix decide qué cachear en EVCache basándose en frecuencia de acceso: catálogo (siempre en caché), perfiles (frecuente), historial completo (nunca en caché).

Resumen T3.4.6: Redis para caché de datos calientes, sesiones y contadores. No para datos críticos ni consultas complejas. Redis complementa la BD, no la sustituye.

T3.4.7 Monitorización y operación

Comando INFO

Secciones del comando INFO

```
INFO server      # versión, uptime, PID
INFO clients     # clientes conectados, bloqueados
INFO memory      # RAM usada, fragmentation ratio, peak memory
INFO persistence # estado RDB/AOF, operaciones de escritura
INFO stats       # ops/sec, cache hit ratio, keyspace hits/misses
INFO replication # estado primary/replica, lag
INFO cpu         # uso de CPU del servidor y hijos
```

Salida de INFO stats:

```
keyspace_hits:12345678 ← Lecturas que encontraron la clave
keyspace_misses:234567  ← Lecturas que NO encontraron la clave
```

Hit rate = hits / (hits + misses) = 12345678 / (12345678 + 234567) = **98.1%**
Un hit rate >95% es saludable. Si baja de 90%, revisa tu TTL o la memoria disponible.

Métricas clave a monitorizar

Métrica	Qué indica	Alerta si...
connected_clients	Conexiones activas	> límite configurado
used_memory	RAM consumida	> 80% de maxmemory
instantaneous_ops_per_sec	Operaciones/segundo	Pico inesperado
keyspace_hits / misses	Ratio de acierto de caché	Hit rate < 80%
expired_keys	Claves expiradas	Muchas expiraciones = TTL bajo
evicted_keys	Claves expulsadas	> 0 (memoria llena)
latest_fork_usec	Tiempo de último fork (RDB o reescritura AOF)	> 1 segundo (bloquea el servidor)

Comando SLOWLOG

Detección de consultas lentas

```
SLOWLOG GET 10      # Últimas 10 consultas lentas
SLOWLOG LEN         # Número de consultas lentas
SLOWLOG RESET       # Limpiar historial
```

Herramientas de monitoreo

- **Redis Insight:** GUI oficial para inspeccionar datos y métricas en tiempo real.
- **redis-cli --latency:** Mide latencia real del servidor.
- **redis-cli --bigkeys:** Encuentra las claves más grandes.
- **redis-cli --stat:** Estadísticas en tiempo real.

GitHub usa SLOWLOG para detectar consultas Redis que superan 10 ms y optimizar el patrón de caché en consecuencia. Esto les permite mantener latencia sub-milisegundo en el 99% de las peticiones.

Resumen T3.4.7: INFO para métricas del servidor. SLOWLOG para consultas lentas. Redis Insight para GUI. Alerta si: evicted_keys > 0, hit rate < 80%, fork > 1s.

Caso real: Discord — Redis para Caché de Chat

Discord usa Redis para caché de mensajes de chat (colores de nombre, badges, bans) con múltiples instancias particionadas por canal.

Arquitectura:

- **Redis:** caché de datos de chat (colores, badges, bans).
- **HAProxy:** balanceo de carga entre instancias Redis.
- **Particionado:** cada canal va a una instancia Redis diferente.
- **Memcached:** alternativa para algunos casos de uso.

El problema inesperado:

Discord descubrió que el cuello de botella **no era Redis**, sino el disco. Las escrituras en disco estaban bloqueando las operaciones Redis.

Solución: "super-disk"

- **Local SSD RAID0:** escritura rápida en disco local.
- **Persistent Disk RAID1:** replicación para persistencia.

Lección: Cuando un sistema va lento, el problema no siempre es donde parece. Discord investigó y descubrió que Redis era rápido, pero el disco de persistencia era el cuello de botella. **Medir antes de optimizar.**

Cuando lo pongas en práctica

- Usa RDB + AOF híbrido en producción: RDB para backups rápidos y AOF para mínimo pérdida de datos.
- Monitoriza INFO memory durante BGSAVE para detectar write amplification por copy-on-write.
- Para Sentinel, configura quórum ≥ 2 para evitar failover fantasma por un Sentinel aislado.
- En Redis Cluster, distribuye claves con prefijos consistentes para evitar hotspots en un solo nodo.
- Replica con al menos 3 nodos para tolerar 1 fallo sin perder disponibilidad.

Preguntas para reflexionar

Pregunta 1

Si Redis usa fork() para RDB y tienes un dataset de 50 GB, ¿qué problema de latencia podrías experimentar? ¿Cómo lo evitarías?

Pregunta 2

¿Qué pasaría si configuras Redis Cluster con solo 3 nodos pero tienes 100.000 claves? ¿Es suficiente?

Pregunta 3

Redis Sentinel failover puede tardar 10-30 segundos. ¿Qué datos de tu aplicación pueden tolerar esa indisponibilidad? ¿Cuáles no? Propón una estrategia para los críticos.



Arquitectura de Datos

4.º Curso, Grado en Ingeniería Informática

Curso académico 2026/2027

Human Language and Accessibility Technologies Group (HULAT).

Departamento de Informática

Universidad Carlos III de Madrid